

The Citadel System

Operating Systems Project — Full Report

Aleix Catà Samaranch

aleix.cata

Pol Pallarès López

pol.pallares

Group G6 · La Salle — Universitat Ramon Llull

Academic Year 2025–2026

Table of Contents

Table of Contents	1
3. Design	2
3.2 Module Structure	3
3.3 Runtime Process Model.....	5
3.4 Communication Protocol	6
3.5 Message Routing and Hops	7
3.6 Alliance System (Pledge)	7
3.7 Product Listing and Trade	8
3.7.1 Listing Products.....	9
3.7.2 Trading	9
3.8 Envoy Concurrency Model.....	10
3.9 Terminal Event Loop	11
3.10 Shared State and Synchronization	12
3.11 Shutdown Sequence.....	13
3.12 System Resources Summary.....	13
4. Data Structures	14
4.1 Route Struct	15
4.2 Maester Struct.....	15
4.3 Product Struct	15
4.4 RemoteInventory Struct.....	15
4.5 TradeItem Struct	16
4.6 Pledge Struct.....	16
4.7 Envoy Struct	16
4.8 Frame Struct.....	16
4.9 PipeMsgHeader Struct	16
5. Observed Problems and Solutions	16
5.1 Memory Leaks (Phase 1).....	16
5.2 Repetitive I/O Code (Phase 1).....	16
5.3 Realm Argument Case Corruption in PLEDGE / PLEDGE RESPOND (Phase 2).....	16
5.4 Missing Route for Relay Tests (Phase 2)	16
5.5 Missing Return in send_pledge_response() Connect-Failure Path (Phase 2/3)	16
5.6 Child Memory Updates Not Visible in Parent (Phase 4)	16
5.7 Zombie Processes (Phase 4).....	16
5.8 PLEDGE Envoy Released Too Early (Phase 4).....	16
5.9 Race Condition on Inventory During Trade (Phase 3/4).....	16
6. Time Estimation	16
Phase 1.....	16
Phase 2.....	16
Phase 3.....	16
Phase 4.....	16

Grand Total — All Phases.....	16
7. Conclusions and Proposals for Improvement	16
7.1 Conclusions.....	16
7.2 Proposals for Improvement.....	16
9. Bibliography.....	16

3. Design

The Citadel System is a distributed application in which each realm is represented by an independent operating system process called the Maester. Every Maester reads its own configuration file, loads a binary inventory, opens a TCP listening socket, and then enters an interactive command loop. From that point on it operates as both a network node that can initiate alliances and trades, and as a router that transparently forwards messages on behalf of other realms. Concurrency is handled at two levels: incoming connections are served by detached pthreads, while outgoing missions are delegated to forked child processes called Envoys. A single select() loop in the parent ties everything together, keeping the terminal responsive at all times.

3.2 Module Structure

The codebase is divided into focused modules, each with a clearly defined single responsibility. This separation makes every module independently testable, keeps implementation details hidden behind header files, and ensures that a bug in one module cannot silently corrupt another. A central io.c utility module wraps all read() and write() calls so that the rest of the codebase never duplicates raw I/O logic, this was also a project requirement, as printf, scanf, and fopen are forbidden.

Module	File(s)	Responsibility
Entry Point & Config	maester.c	Program startup, argument validation, signal installation, Maester configuration reading (realm name, routes, Envoy count), and top-level cleanup on exit
Terminal	terminal/terminal.c, commands.c	Main interactive loop using select(), and command parsing and routing to feature handlers
Inventory	inventory/inventory.c	Reads and writes the binary stock.db, and manages the local Product array and trade menu
Pledge	pledge/pledge.c	Maintains the alliance table, tracks status, ally IP/port, and timestamps for timeouts
Network	network/network.c, protocol.c	Creates and manages TCP sockets, builds, serialises, validates, and dispatches frames
Handler Dispatcher	handlers/message_handler.c	Dispatches incoming frames to the correct handler, provides shared parsing helpers
Pledge Handler	handlers/pledge_handler.c	Implements incoming alliance frame handling (0x01 and 0x03): sigil reception, MD5 check, pledge recording, and response dispatch
List Handler	handlers/list_handler.c	Handles incoming LIST REQUEST frames (0x11): verifies alliance, serialises stock, sends LIST HEADER (0x12) and LIST DATA (0x13) frames
Order Handler	handlers/order_handler.c	Handles incoming trade ORDER frames (0x14): receives order file, aggregates lines, validates stock atomically, replies ORDER RESPONSE (0x16)

Module	File(s)	Responsibility
Transfer	transfer/sigil.c	Sends and receives files in chunked frames, verifies MD5 end-to-end integrity using fork + execvp(md5sum)
Relay	transfer/relay.c	Forwards PLEDGE frames through intermediate hop nodes without modifying ORIGIN or DESTINATION
Envoy	envoy/envoy.c	Manages Envoy slots, it forks child processes for outgoing missions; reads pipe results and applies them to parent state
I/O	utils/io.c	Wraps read()/write() into safe helpers: read_line, write_all, etc.

Two structural decisions are worth noting. First, the previously monolithic message handler was split into four focused files under the handlers/ directory: message_handler.c acts as the dispatcher and hosts shared helper functions, while pledge_handler.c, list_handler.c, and order_handler.c each own one protocol concern. This makes it straightforward to add a new frame type without touching unrelated code. Second, sigil.c and relay.c were relocated from network/ into a dedicated transfer/ directory, separating the concerns of raw socket management from file-transfer and hop-forwarding logic.

The diagram below shows the full dependency graph. Arrows represent 'uses' relationships. The cylindrical node at the bottom represents the file system resources such as: configuration files, stock databases, sigil images. Access to these files is funnelled exclusively through the Entry Point, Inventory, and Handler modules, keeping I/O concerns isolated.

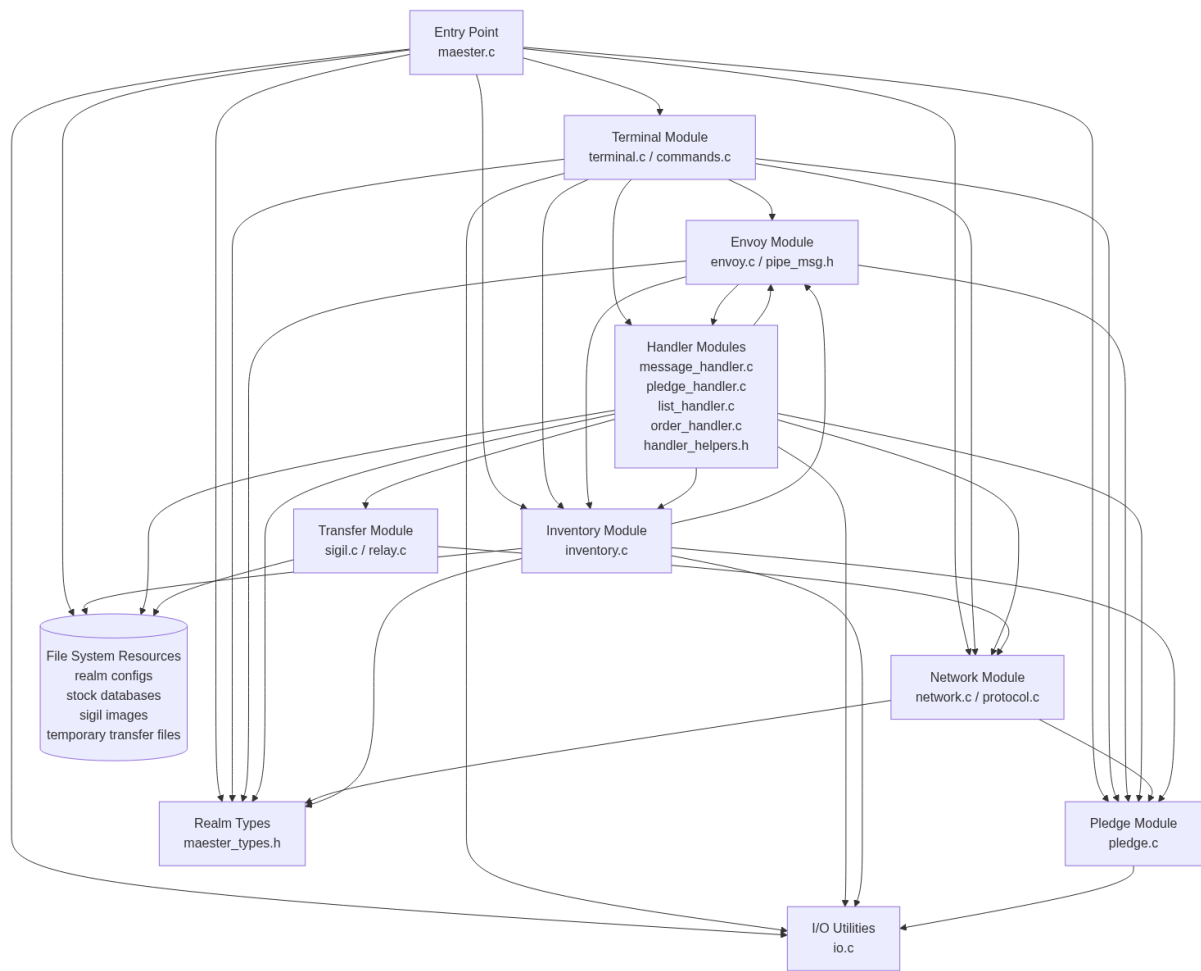


Figure 1 — Directory and file structure of the Citadel System codebase

3.3 Runtime Process Model

At runtime, a single Maester process owns all long-lived state: the configuration, the pledge table, the local inventory, the remote inventory cache, and the Envoy slot array. Two categories of concurrent workers are created dynamically as needed:

- Detached pthreads: one per accepted incoming TCP connection. Because incoming frames must immediately update parent-owned state such as recording an incoming alliance request or reducing seller stock. Threads are used so that updates happen directly in shared memory under mutex protection, with no extra IPC required.
- Forked Envoy children: one per outgoing mission such as PLEDGE, LIST PRODUCTS, TRADE and PLEDGE RESPOND. Processes provide isolation: a blocked or crashed child cannot corrupt the parent. Because fork() creates separate address spaces, any state the child computes must be sent back to the parent through a result pipe.

The central select() loop is the heart of the system. It simultaneously monitors stdin, the server socket, a self-pipe, and the read ends of all active Envoy result pipes. No single event source can block the others.

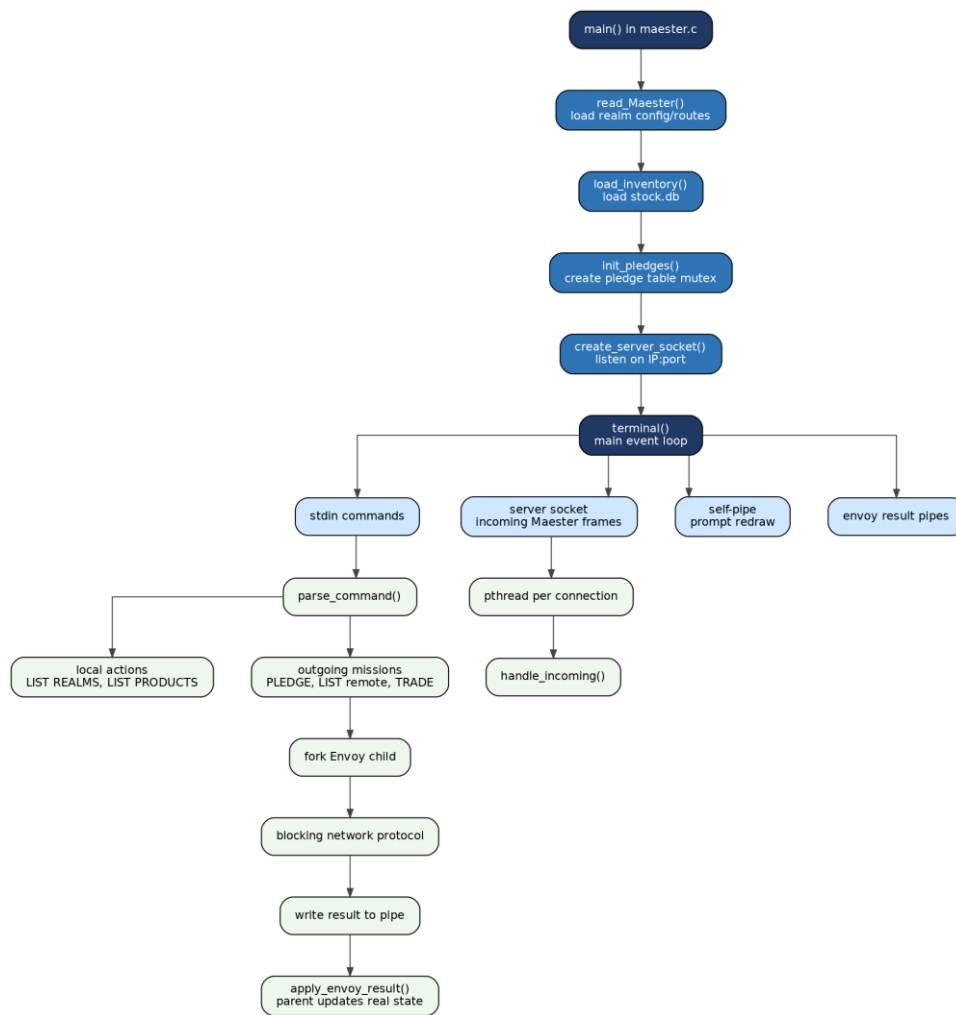


Figure 2 — Startup sequence and event sources of the Maester process

3.4 Communication Protocol

All inter-Maester communication uses TCP sockets with a custom binary frame protocol. Every frame is exactly 320 bytes, regardless of message type. This fixed size simplifies socket I/O: `recv_frame()` and `send_frame()` always read or write precisely `sizeof(Frame)` bytes, eliminating the need to parse variable-length headers.

Field	Size	Description
Type	1 byte	Frame identifier (e.g., 0x01 = alliance header, 0x11 = product list request)
Origin	20 bytes	IP:Port of the originating realm (not the hop)
Destination	20 bytes	Name of the destination realm
Data Length	2 bytes	Number of valid bytes in the Data field
Data	275 bytes	Payload; padded with zeros if smaller than maximum
Checksum	2 bytes	Sum of all other bytes modulo 65536

Every received frame is validated against its checksum before processing. If the check fails, the receiver immediately sends a NACK (TYPE 0x69) and discards the frame. This per-frame validation catches transport corruption early. A separate end-to-end MD5 check is applied to complete file transfers (sigils, product lists, trade orders), providing application-layer integrity on top of the per-frame transport check.

3.5 Message Routing and Hops

When a Maester receives a frame, it first checks whether it is the DESTINATION. If not, it acts as a transparent relay: it validates the checksum, looks up the DESTINATION in its route table, and forwards the frame to the next node without modifying the ORIGIN or DESTINATION fields. This preserves end-to-end addressing across an arbitrary number of hops.

Route lookup proceeds as follows: if a specific entry for the destination exists in the configuration file, that IP:Port is used. If the destination appears with wildcard address `*.*.*.*`, or if no specific entry is found, the DEFAULT route is used. If there is no DEFAULT route, the relay sends a TYPE 0x21 UNKNOWN_REALM error back to the origin so it does not wait forever.

Once two realms establish an alliance, each stores the other's direct IP:Port in its pledge table. All subsequent communication between those realms is direct, so no intermediate nodes are traversed. The diagram below shows how incoming connections are handled and how the frame type determines which handler is invoked.

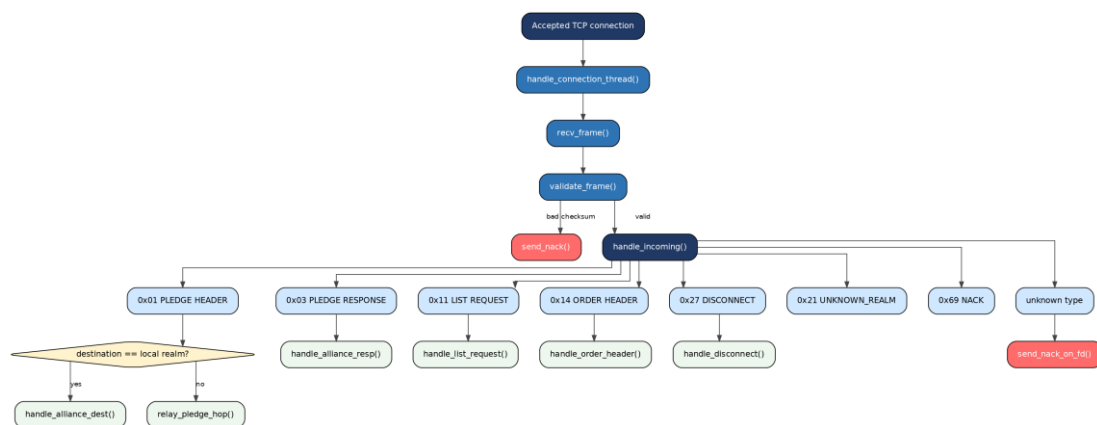


Figure 3 — Incoming connection handling: per-thread dispatch and frame type routing

3.6 Alliance System (Pledge)

Alliances are established through a five-step handshake. The initiating realm dispatches an Envoy child that manages the entire exchange: it validates the sigil file, computes its MD5, and sends the full sequence of frames. The destination processes the incoming request in a connection thread, stores an incoming pending pledge, and waits for the user to respond.

1. The origin sends TYPE 0x01 (ALLIANCE HEADER) with realm name, sigil filename, file size, and MD5 hash.
2. The destination replies TYPE 0x31 (ACK FILE: OK) to authorise the transfer.
3. The origin sends the sigil image in one or more TYPE 0x02 (SIGIL DATA) frames, chunked to fit the 275-byte payload.
4. The destination verifies the reconstructed file's MD5 and replies TYPE 0x32 (ACK MD5: CHECK_OK or CHECK_KO). On failure the transfer is aborted.

5. After the destination user issues PLEDGE RESPOND ACCEPT or REJECT, a response Envoy sends TYPE 0x03 (ALLIANCE RESPONSE). Crucially, the ORIGIN field of this frame carries the destination's own IP:Port, letting the origin learn the ally's direct address. The origin replies TYPE 0x31 ACK, or TYPE 0x69 NACK if the 120-second timeout has already expired.

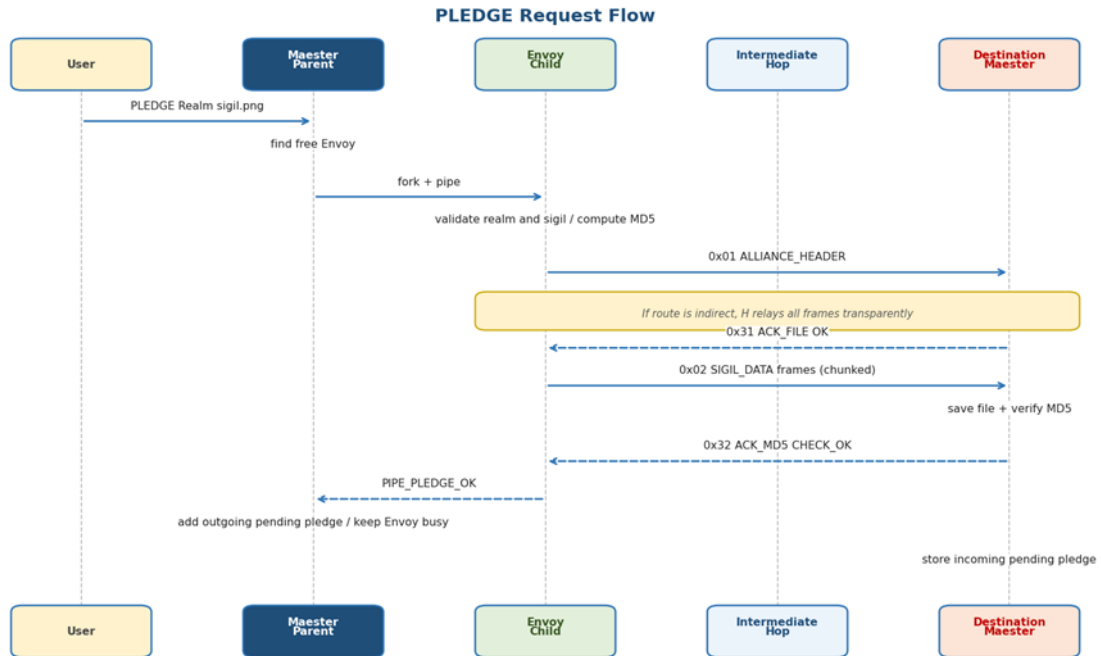


Figure 4 — PLEDGE full flow: sigil transfer via optional hops, and direct response back to origin

The timeout mechanism prevents one-sided alliances. The origin's pledge table records the time at which the outgoing pledge was sent. Every `select()` iteration calls `check_pledge_timeouts()`, which marks any pledge that has been `OUTGOING_PENDING` for more than 120 seconds as `FAILED`. If the destination later sends `ACCEPT`, the origin sees the pledge is no longer pending and sends `NACK` instead of `ACK`, so neither side believes an alliance exists.

3.7 Product Listing and Trade

Trade between allied realms is a two-stage process: first the buyer requests the seller's product list, then the buyer builds and sends a shopping order. Both stages transfer binary files over the network using the same header-plus-data frame pattern as the sigil transfer.

3.7.1 Listing Products

`LIST PRODUCTS <REALM>` dispatches an Envoy child that sends TYPE 0x11 (LIST REQUEST). The seller verifies the requester is an ally, serialises its `stock.db` file, computes its MD5, and sends a TYPE 0x12 (LIST HEADER) followed by 0x13 (LIST DATA) frames. After verifying the MD5, the buyer parses the Product structs and sends the array back to the parent through the result pipe. The parent stores these products in a per-realm remote inventory cache.

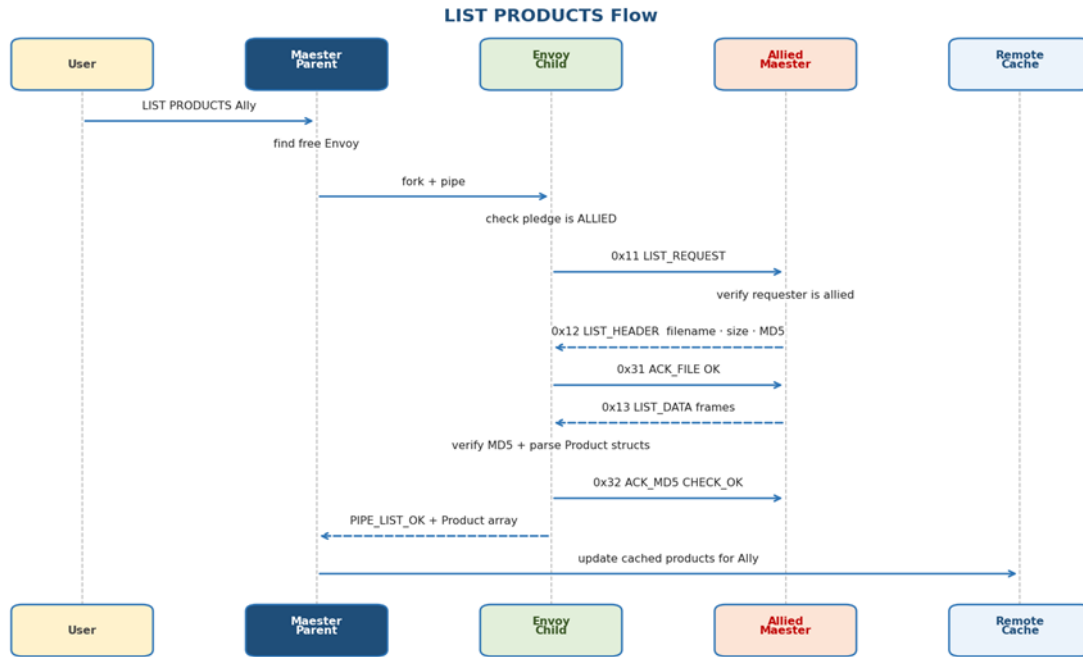


Figure 5 — LIST PRODUCTS flow: request, file transfer, MD5 verification, and parent cache update

The cache update must go through the pipe because `fork()` produces copy-on-write memory isolation: any changes the child makes to its copy of the inventory are invisible to the parent. The pipe is the only correct mechanism to transfer structured data back.

3.7.2 Trading

START TRADE <REALM> reserves an Envoy slot immediately, before the trade menu is even shown. This prevents the system from starting more missions than available Envoys. The terminal takes a snapshot of the remote cache under `g_data_mutex` so the menu has a stable product list even if another thread updates the cache concurrently. The user can ADD and REMOVE items, then SEND or CANCEL.

On SEND, the Envoy child creates a text order file, computes its MD5, and sends a TYPE 0x14 (ORDER HEADER) followed by 0x15 (ORDER DATA) frames. The seller verifies the MD5, aggregates duplicate order lines, then validates and applies the stock change atomically under `g_data_mutex`. It replies TYPE 0x16 (ORDER RESPONSE) with OK or REJECT&reason (OUT_OF_STOCK or UNKNOWN_PRODUCT). On acceptance, both inventories are updated and persisted to `stock.db`.

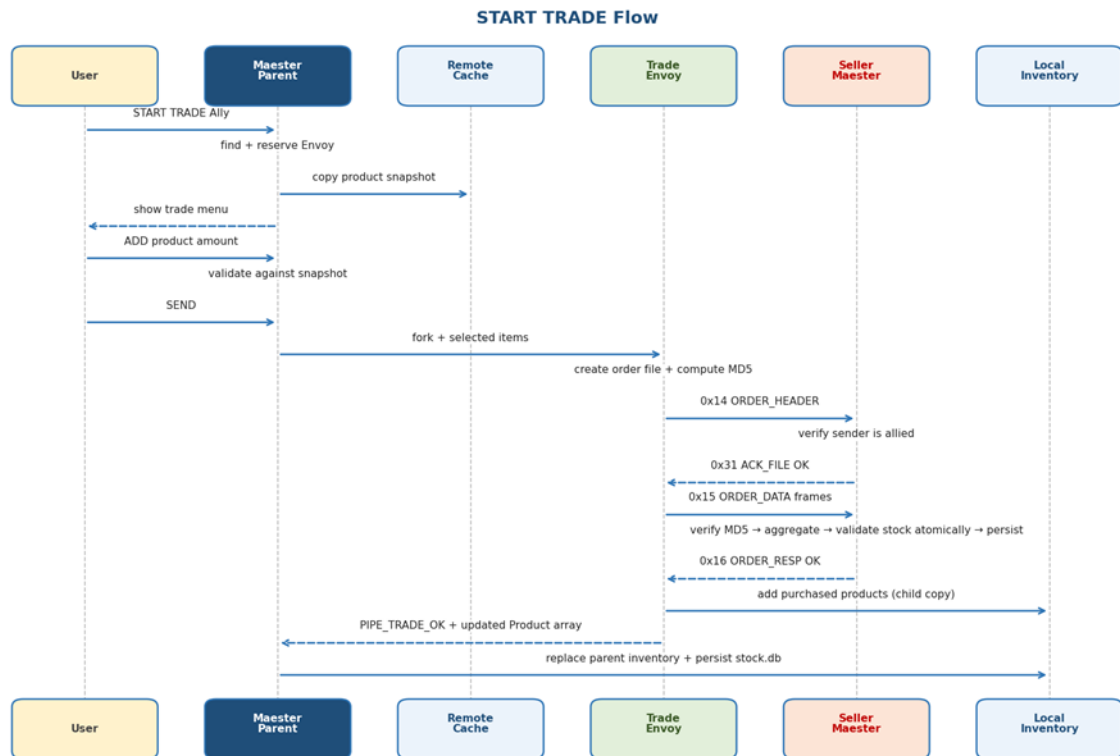


Figure 6 — START TRADE flow: order creation, file transfer, atomic stock validation, and dual inventory update

3.8 Envoy Concurrency Model

Each Maester has a fixed pool of Envoy slots, the size of which is read from the configuration file. A slot is a small struct holding the mission type, the target realm name, the child PID, and the pipe read descriptor. When the user triggers an outgoing command, the Maester finds a FREE slot, marks it, creates a pipe, and forks. The child performs the full network operation, which may block on socket I/O, file transfer, or MD5 verification, without affecting the parent terminal. The child writes a PipeMsgHeader to the pipe, then exits. The parent's select() loop detects data on the pipe read end and calls apply_envoy_result(), which updates the pledge table or inventory accordingly and releases the slot.

The state diagram below shows every possible Envoy state transition. The PLEDGE WAITING state is the most important subtlety: a PLEDGE Envoy is not released when its child exits after delivering the sigil. It remains reserved until the diplomatic outcome arrives, ACCEPT, REJECT, or 120-second timeout, ensuring ENVOY STATUS always accurately reflects active missions.

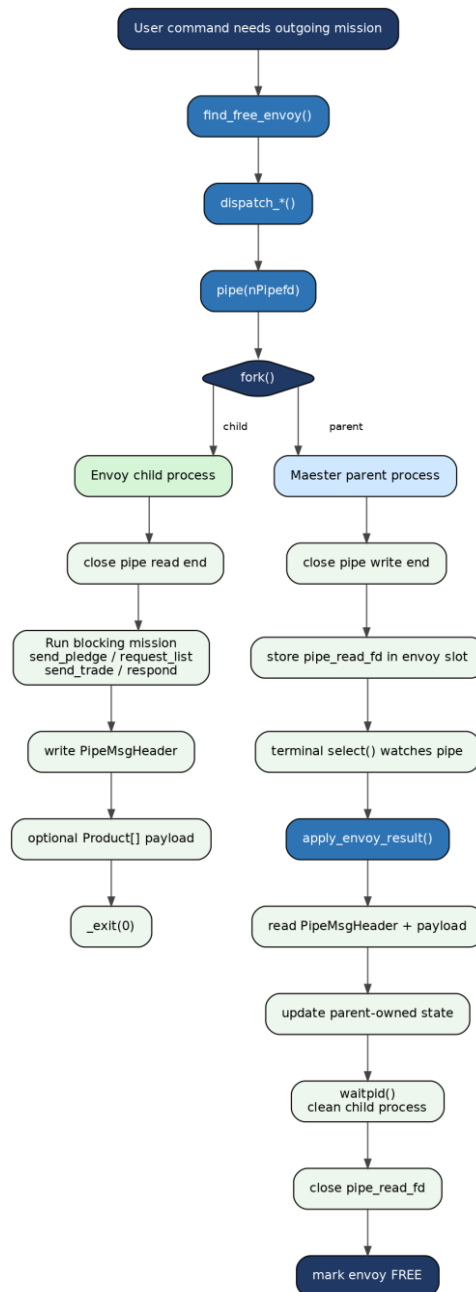


Figure 7 — Envoy fork/pipe lifecycle: from mission dispatch to parent state update

3.9 Terminal Event Loop

The terminal loop uses `select()` to multiplex four categories of file descriptors. On each iteration it rebuilds the `fd_set`, calls `select()`, and then processes each ready source in order: incoming connections first, then self-pipe notifications from background threads, then Envoy pipe results, and finally stdin commands. Before looping back it checks pledge timeouts and calls `check_pledge_envoys()` to free any Envoy whose mission outcome has been resolved.

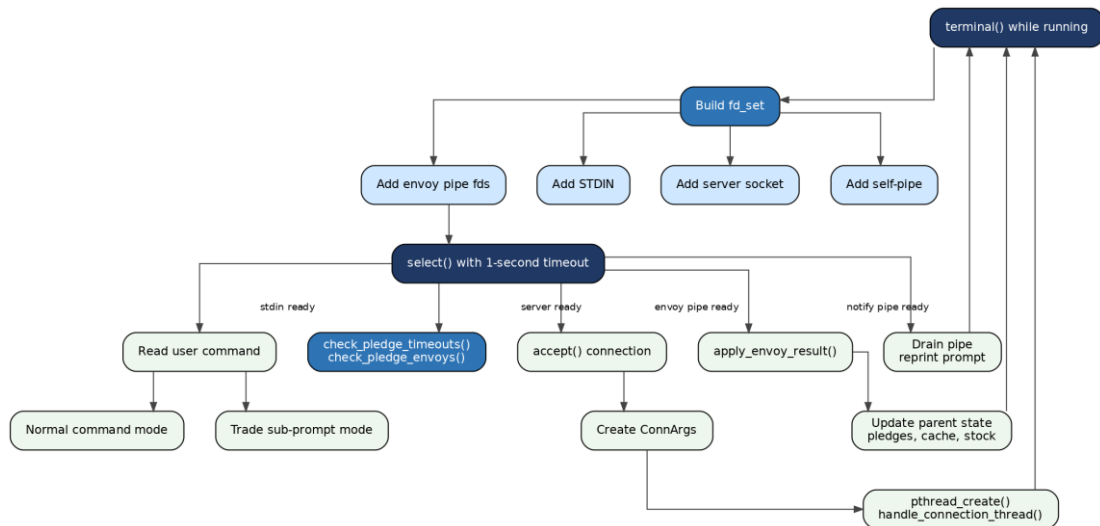


Figure 8 — Terminal event loop: `select()` multiplexing all event sources

3.10 Shared State and Synchronization

Three concurrent actors can read or write shared state at any time: the Maester's main terminal loop, incoming connection threads, and Envoy result handlers. Two independent mutexes protect the two distinct data domains:

- `g_pledge_mutex` (recursive): protects the pledge table. It is recursive because some higher-level functions that hold the lock call helper functions that also acquire it, a standard mutex would self-deadlock in those paths.
- `g_data_mutex`: protects the local product array and the remote inventory cache. Incoming trade handlers reduce seller stock; Envoy result handlers update the buyer's cache or inventory; the terminal reads both for LIST PRODUCTS and START TRADE. The final stock validation and deduction happen atomically inside the same lock to avoid a time-of-check / time-of-use race condition.

Forked Envoy children communicate back to the parent exclusively through the result pipe. They do not hold any locks and do not share memory with the parent after the fork, any changes they make to their private copy are invisible to the parent until explicitly transmitted through the pipe.

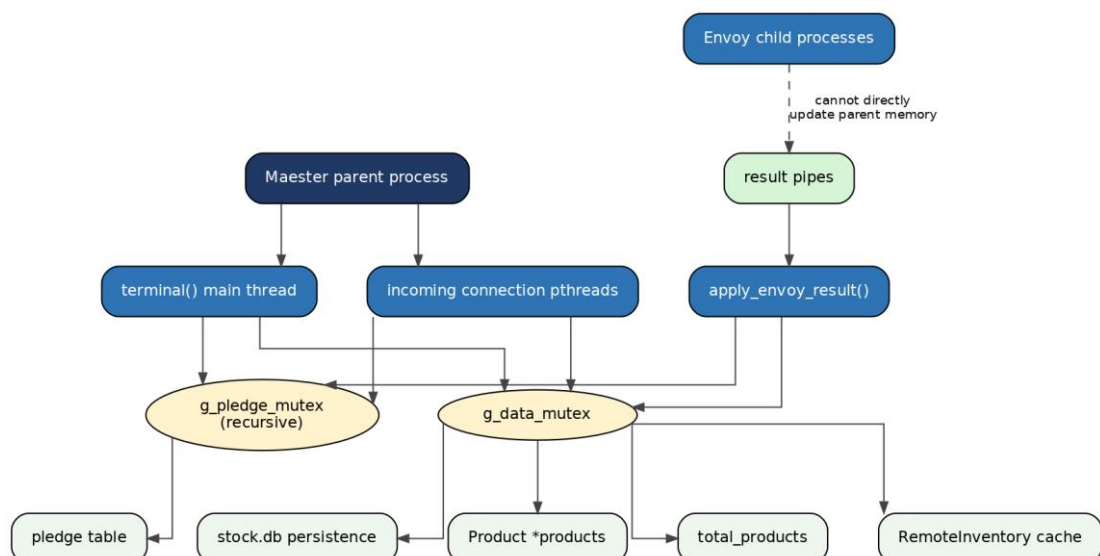


Figure 9 — Shared state and synchronization: two mutexes and pipe-based child-to-parent IPC

3.11 Shutdown Sequence

Shutdown is triggered either by the EXIT command or by CTRL+C (SIGINT). The signal handler only sets a flag and closes STDIN, it does not perform any cleanup itself, because complex operations such as mutex acquisition, socket writes, and malloc/free are not async-signal-safe. The terminal loop detects the flag, exits, and executes the full teardown in ordinary program flow.

The teardown order is intentional: connection threads and Envoy children are waited for before any shared memory is freed, preventing use-after-free. DISCONNECT frames (TYPE 0x27) are broadcast to all currently allied realms so they can mark the alliance inactive before the process disappears.

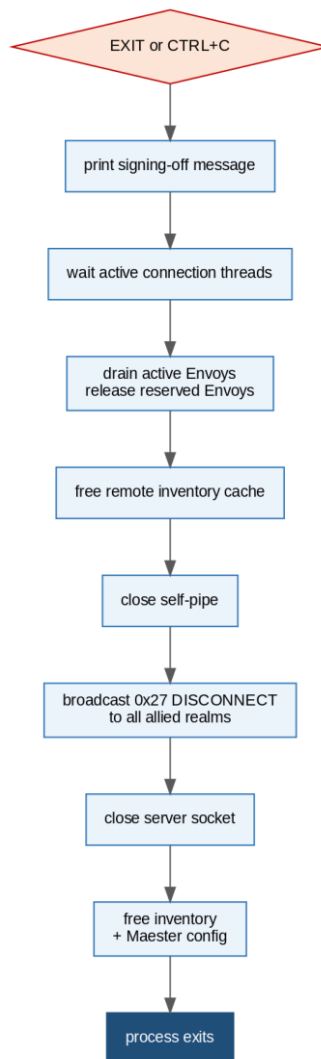


Figure 10 — Shutdown sequence: ordered resource release and DISCONNECT broadcast to all allies

3.12 System Resources Summary

Resource	Usage	Justification
Signals (SIGINT)	CTRL+C sets stop_flag and closes STDIN, cleanup runs in normal code	Signal handlers must be async-signal-safe, only flag manipulation is safe inside them
Binary file I/O (read/write)	Configuration parsing and binary inventory access	printf/scanf/fopen are forbidden by the specification, all I/O uses read() and write()
TCP Sockets	One server socket per Maester, one outgoing socket per mission	Maesters may run on separate machines, TCP provides reliable ordered delivery essential for multi-frame transfers and ACK/NACK logic
select()	Multiplexes stdin, server socket, self-pipe, and Envoy result pipes	Allows the Maester to remain responsive to all event sources simultaneously without busy-waiting
pthreads (detached)	One thread per accepted incoming connection	Incoming frames must update shared parent state, threads share address space, making updates direct and efficient under mutex protection
Recursive Mutex (g_pledge_mutex)	Protects the pledge table	Accessed concurrently by terminal, connection threads, and Envoy result handlers, recursive to avoid self-deadlock in nested call paths
Mutex (g_data_mutex)	Protects the inventory array and remote inventory cache	Multiple concurrent actors read/write product data, the mutex also serialises the stock validation+update pair to prevent race conditions
Condition Variable (g_conn_cond)	Shutdown: wait until all active connection threads have finished	Prevents freeing shared resources while threads are still using them
fork()	Creates md5sum subprocess for file integrity, creates Envoy child processes	Provides process-level isolation for Envoy missions, required by Phase 4 specification
pipe()	Captures md5sum stdout, carries structured Envoy results to the parent	After fork, memory changes in the child are invisible to the parent, pipes are the correct mechanism to return structured data
dup2()	Redirects child stdout to the pipe read end for md5sum capture	Allows the parent to read md5sum output through a file descriptor without using popen
execvp()	Replaces the md5sum child image with the system md5sum binary	Avoids implementing MD5 in C, as required by the specification
waitpid()	Reaps md5sum subprocesses and Envoy children	Prevents zombie processes, called in apply_envoy_result() and wait_all_envoys() at shutdown

4. Data Structures

4.1 Route Struct

```
typedef struct {
    char *maester;    // Realm name (dynamically allocated)
    char *ip;         // IP address (dynamically allocated)
    int  port;        // Port number
} Route;
```

Dynamic strings avoid wasting memory on fixed-size buffers and allow arbitrarily long realm names and IP addresses. The port is an int, matching the range required by the protocol. Route entries are stored in a heap-allocated array inside the Maester struct and freed individually in `free_Maester()`.

4.2 Maester Struct

```
typedef struct {
    char *realm_name;    // Name of this realm
    char *user_dir;      // Path to realm's file directory
    int  envoy_count;    // Number of Envoy slots
    char *listen_ip;     // IP where this Maester listens
    int  listen_port;    // Port where this Maester listens
    Route *routes;       // Dynamic array of routing entries
    int  route_count;    // Number of entries in routes[]
    char *stock_path;    // Path to the binary stock.db file
} Maester;
```

All string fields are dynamically allocated to match the exact length of each value read from the configuration file. The routes array is a contiguous heap allocation sized to `route_count`, enabling $O(n)$ route lookup. The Maester struct is the single authoritative source of realm identity and topology. Its definition and lifecycle functions live in `maester.c` at the project root, the struct declaration is in `realm/maester_types.h` so it can be included by any module without pulling in unrelated code.

4.3 Product Struct

```
typedef struct {
    char name[100];    // Product name (fixed-size – required by binary file
format)
    int  amount;       // Stock quantity
    float weight;      // Weight per unit
} Product;
```

Fixed-size fields are mandatory: because the inventory is stored as a binary file, `read()` must know exactly how many bytes constitute one record. Using dynamic pointers would break direct binary deserialisation. The same struct is used for both local stock and for the remote inventory snapshots cached after LIST PRODUCTS calls.

4.4 RemoteInventory Struct

```
typedef struct {
    char    realm_name[REALM_NAME_SIZE]; // Remote realm key
    Product *products;                   // Cached Product[] snapshot
    int     count;                       // Number of cached products
} RemoteInventory;
```

One RemoteInventory entry is maintained per allied realm. After every successful LIST PRODUCTS exchange the Envoy child sends the parsed Product array back to the parent through the result pipe,

the parent calls `ri_set_products()` under `g_data_mutex` to update the canonical cache. The cached snapshot is what the trade menu displays when the user runs `START TRADE`.

4.5 TradeItem Struct

```
typedef struct {
    char *name;    // Product name (dynamically allocated)
    int  amount;   // Requested quantity
} TradeItem;
```

Used during the interactive trade session to accumulate the shopping list before serialisation to the order file. The dynamic string allows product names of arbitrary length. All `TradeItem` entries are freed in `trade_session_free()` once the order has been dispatched or cancelled.

4.6 Pledge Struct

```
typedef struct {
    char  realm_name[REALM_NAME_SIZE]; // Name of the allied/pending realm
    int   status;    // OUTGOING_PENDING | INCOMING_PENDING | ALLIED | FAILED |
    INACTIVE
    char  ip[IP_SIZE]; // Direct IP of ally (filled on alliance acceptance)
    int   port;        // Direct port of ally
    time_t sent_time;  // When the outgoing pledge was sent (for timeout)
} Pledge;
```

The fixed-size `realm_name` simplifies array management, the pledge table is a static array of `MAX_PLEDGES` entries protected by `g_pledge_mutex`. The `ip` and `port` fields are populated when a `0x03 ALLIANCE RESPONSE` is received, enabling direct communication after alliance formation. `sent_time` enables the 120-second timeout check performed on every `select()` iteration.

4.7 Envoy Struct

```
typedef struct {
    int      id; // 1-based display ID
    EnvoyMission mission; // FREE | PLEDGE | PLEDGE_WAITING | LIST | TRADE
| RESPOND
    char  realm[50]; // Target realm for the current mission
    pid_t pid;      // PID of the forked child (-1 if none)
    int   pipe_read_fd; // Read end of the result pipe (-1 when unused)
} Envoy;
```

Each `Envoy` slot is permanently allocated at startup. The `pipe_read_fd` descriptor is included in the `select()` `fd_set`, so the parent wakes immediately when a child result is available, with no polling required. The `PLEDGE_WAITING` state keeps the slot reserved after the sigil child exits but before the `ACCEPT/REJECT` response arrives.

4.8 Frame Struct

```
typedef struct {
    uint8_t type; // Frame type identifier (MSG_* constant)
    char  origin[20]; // IP:Port of originating realm
    char  destination[20]; // Name of destination realm
    short data_length; // Bytes used in data[]
    char  data[275]; // Payload (padded with zeros)
    short checksum; // Sum of all other bytes % 65536
} Frame; // sizeof(Frame) == 320
```

This `Frame` struct represents one fixed-size protocol message of exactly 320 bytes, containing the message type, sender address, destination realm, payload length, payload data, and checksum.

Every Maester sends and receives communication through these frames, and the checksum lets the receiver verify that the frame was not corrupted in transit.

4.9 PipeMsgHeader Struct

```
typedef enum {
    PIPE_PLEDGE_OK, PIPE_PLEDGE_FAIL,
    PIPE_LIST_OK,   PIPE_LIST_FAIL,
    PIPE_TRADE_OK,  PIPE_TRADE_FAIL,
    PIPE_RESPOND_DONE
} PipeMsgType;

typedef struct {
    PipeMsgType type;      // PIPE_* result type
    char        realm[50]; // Realm associated with the completed mission
    int         count;     // Product entries that follow (LIST_OK / TRADE_OK)
} PipeMsgHeader;
```

The PipeMsgHeader is written by every Envoy child before exiting. For LIST_OK and TRADE_OK, count Product structs follow immediately after the header, allowing the parent to reconstruct the full result in a single read sequence using read_all(). The PipeMsgType enum makes the result type explicit and exhaustive, the compiler will warn if a new mission type is added without updating the switch in apply_envoy_result().

5. Observed Problems and Solutions

5.1 Memory Leaks (Phase 1)

Dynamic allocation for route strings and product names led to leaked memory when the `free()` was placed in a different function than the `malloc()`. Diagnosed using Valgrind, which identified every leaked allocation with its call stack. The solution was to consolidate all cleanup into a single exit path (`free_Maester + free_inventory`), ensuring that every heap allocation is released in reverse creation order.

5.2 Repetitive I/O Code (Phase 1)

The initial implementation had duplicated read/write patterns scattered across multiple files, making maintenance difficult and increasing the risk of inconsistent error handling. The solution was to introduce `io.c` with helper functions (`read_line`, `write_str`, `write_all`) that encapsulate all I/O logic, reducing the rest of the codebase to single-function calls.

5.3 Realm Argument Case Corruption in PLEDGE / PLEDGE RESPOND (Phase 2)

The command parser was copying the entire remainder of the input string into the realm argument instead of just the realm token, causing realm name lookups to fail silently. The fix was to change `strndup(orig_w2, strlen(w2))` to `strndup(orig_w2, token_len)`, ensuring only the correct token length is copied.

5.4 Missing Route for Relay Tests (Phase 2)

In the test topology, Realm C lacked a direct entry for Realm A in its routing table, so pledges relayed via Realm B could not be forwarded. Adding the missing route entry resolved all multi-hop relay tests.

5.5 Missing Return in `send_pledge_response()` Connect-Failure Path (Phase 2/3)

When the outgoing socket connection failed inside `send_pledge_response()`, the function continued execution without returning `PLEDGE_FAILED`, causing unpredictable behaviour. The fix was to add an explicit return `PLEDGE_FAILED` immediately after logging the connection error.

5.6 Child Memory Updates Not Visible in Parent (Phase 4)

After introducing fork-based Envoys, LIST PRODUCTS updates in the child process were not reflected in the parent's remote inventory cache. This is a fundamental consequence of copy-on-write memory isolation after `fork()`. The solution is the pipe protocol: the child serialises its result into a `PipeMsgHeader` followed by Product structs, and the parent reconstructs the cache by reading from the pipe.

5.7 Zombie Processes (Phase 4)

Early Envoy children were not always reaped with `waitpid()`, leaving zombie entries in the process table. The solution was to always call `waitpid(child_pid)` inside `apply_envoy_result()` once the pipe result has been read, and to call `wait_all_envoys()` during shutdown to drain remaining children.

5.8 PLEDGE Envoy Released Too Early (Phase 4)

An Envoy was initially released as soon as the child sent the sigil and exited, before the diplomatic response (ACCEPT/REJECT) had arrived. This meant the Envoy slot became free while the mission was still logically active. The fix was to keep the slot in `ENVOY_PLEDGE` state until `check_pledge_envoys()` confirms the pledge is no longer `OUTGOING_PENDING`.

5.9 Race Condition on Inventory During Trade (Phase 3/4)

Two simultaneous incoming trade orders could each read the same stock value, both pass the stock check, and both deduct from inventory — overselling by a factor of two. The solution was to perform stock validation and deduction atomically under `g_data_mutex`, re-reading the current product pointer inside the lock (time-of-check / time-of-use protection).

6. Time Estimation

Estimated hours spent per category and per student across all four phases.

Phase 1

Category	Aleix (h)	Pol (h)	Combined (h)
Research	2	2	4
Design	5	3	8
Implementation	4	3	7
Testing	3	1	4
Documentation	1	7	8
TOTAL	15	16	31

Phase 2

Category	Aleix (h)	Pol (h)	Combined (h)
Research	3	3	6
Design	6	5	11
Implementation	8	8	16
Testing	4	3	7
Documentation	2	2	4
TOTAL	23	21	44

Phase 3

Category	Aleix (h)	Pol (h)	Combined (h)
Research	2	2	4
Design	4	4	8
Implementation	9	9	18
Testing	5	4	9
Documentation	2	2	4
TOTAL	22	21	43

Phase 4

Category	Aleix (h)	Pol (h)	Combined (h)
Research	3	2	5
Design	5	4	9
Implementation	10	9	19
Testing	5	5	10
Documentation	2	3	5
TOTAL	25	23	48

Grand Total — All Phases

Category	Aleix (h)	Pol (h)	Combined (h)
Research	10	9	19
Design	20	16	36
Implementation	31	29	60
Testing	17	13	30
Documentation	7	14	21
TOTAL	85	81	166

7. Conclusions and Proposals for Improvement

7.1 Conclusions

The Citadel System successfully implements a distributed multi-process communication network across all four phases. The incremental structure proved very effective: each phase introduced a clearly scoped set of new OS mechanisms on top of a stable, already-tested foundation.

The most significant design insight was the distinction between shared-memory concurrency (threads for incoming connections) and message-passing concurrency (fork + pipe for Envoys). The two approaches complement each other naturally in a server that must handle both roles simultaneously. Using threads for incoming connections made state updates simple and direct; using fork for Envoy missions provided process-level isolation and satisfied the Phase 4 specification.

The select()-based event loop proved to be the correct architectural choice: it allowed the process to remain responsive to user input, incoming connections, and Envoy results simultaneously, without spinning or blocking on any single source. Building the full binary protocol from scratch, including frame serialisation, checksum validation, ACK/NACK logic, multi-frame file transfers, and MD5 verification, gave deep practical understanding of how real distributed protocols handle reliability at the application layer.

7.2 Proposals for Improvement

- Persistent alliance state: all alliance information is lost when a Maester restarts. A binary file written at every alliance update would allow realms to resume their diplomatic relationships after a crash.
- Dynamic routing updates: after an alliance is formed, the ally's IP:Port is stored locally but no other realm learns about it. A route-advertisement protocol similar to RIP could allow the network to self-organise without static routing tables.
- Encryption: sigils, product lists, and trade orders are sent in plaintext. Adding TLS would protect trade secrets and prevent forged frames from impersonating allied realms.
- Retry logic: if the first connection attempt fails, the mission is marked failed. Implementing exponential back-off with a configurable retry count would make the system more robust against transient network failures.
- Larger data field: the fixed 320-byte frame limits payloads to 275 bytes. Supporting fragmented data with a sequence number would allow longer strings and file metadata without breaking the fixed-frame model.

9. Bibliography

- [1] "Valgrind Quick Start Guide," Valgrind.org. [Online]. Available: <https://valgrind.org/docs/manual/quick-start.html> [Accessed: May 2026].
- [2] GeeksforGeeks, "How to Use Make Utility to Build C Projects?", GeeksforGeeks, Apr. 30, 2020. [Online]. Available: <https://www.geeksforgeeks.org/c/how-to-use-make-utility-to-build-c-projects/> [Accessed: Mar. 2026].
- [3] Rigon, "Read binary data (from file) into a struct," Stack Overflow, Oct. 21, 2010. [Online]. Available: <https://stackoverflow.com/questions/3988709/read-binary-data-from-file-into-a-struct> [Accessed: Mar. 2026].
- [4] W. R. Stevens, B. Fenner, and A. M. Rudoff, Unix Network Programming, Volume 1: The Sockets Networking API, 3rd ed. Addison-Wesley, 2003.
- [5] W. R. Stevens, Advanced Programming in the UNIX Environment, 3rd ed. Addison-Wesley, 2013.
- [6] "The Open Group Base Specifications Issue 7 — select()," IEEE/The Open Group, 2018. [Online]. Available: <https://pubs.opengroup.org/onlinepubs/9699919799/functions/select.html>
- [7] "The Open Group Base Specifications Issue 7 — fork()," IEEE/The Open Group, 2018. [Online]. Available: <https://pubs.opengroup.org/onlinepubs/9699919799/functions/fork.html>
- [8] "Linux man page — pipe(2)," The Linux man-pages project. [Online]. Available: <https://man7.org/linux/man-pages/man2/pipe.2.html>
- [9] "Linux man page — pthread_create(3)," The Linux man-pages project. [Online]. Available: https://man7.org/linux/man-pages/man3/pthread_create.3.html
- [10] "md5sum — GNU Coreutils," GNU Project. [Online]. Available: <https://www.gnu.org/software/coreutils/manual/coreutils.html#md5sum-invocation>